



**YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMICAL AND METALLURGY
DEPARTMENT OF MATHEMATICAL ENGINEERING**

Graduation Project

Developing a Social Media Platform with Real-Time AI Content Creation

Assoc. Prof. Birol ASLANYÜREK

22058023, Kuzey SINAY

Istanbul, 2026

**© All rights to this thesis are reserved by the Department of Mathematical Engineering,
Yıldız Technical University.**

TABLE OF CONTENTS

SYMBOL LIST	iv
ABBREVIATION LIST	v
FIGURE LIST	vi
TABLE LIST	vii
ABSTRACT	ix
1. INTRODUCTION	1
1.1. Background and Motivation	1
1.2. Problem Definition, Aim And Scope	1
1.2.1. <i>Problem Definition</i>	1
1.2.2. <i>Aim of the Study</i>	2
1.2.3. <i>Scope of the Study</i>	2
1.3. Review of Related Works	2
2. PROPOSED METHODOLOGY	4
2.1. Key Concepts And Theoretical Framework	4
2.2. System Architecture	5
2.3. Component Design	5
3. IMPLEMENTATION AND EXPERIMENTAL SETUP.....	7
3.1 Development Environment and Tools	7
3.2 Backend Implementation	7
3.3 iOS Client Implementation	8
4. CURRENT PROGRESS AND NEXT STEPS.....	10
4.1. Achievements.....	10
4.2. Future Work	11
REFERENCES	12
Resume	13

SYMBOL LIST

T	Streaming token (partial text chunk emitted by the language model)
N	Number of posts seen in the current session (seen_count)
l	Set of user-provided interest strings
P	Generated post, consisting of a title and a body
E	SSE event frame, consisting of an event name and a JSON data payload

ABBREVIATION LIST

AI	Artificial Intelligence
API	Application Programming Interface
ATS	App Transport Security
CORS	Cross-Origin Resource Sharing
GCD	Grand Central Dispatch
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
iOS	iPhone Operating System
JSON	JavaScript Object Notation
LLM	Large Language Model
MVVM	Model–View–ViewModel
NLP	Natural Language Processing
REST	Representational State Transfer
SDK	Software Development Kit
SSE	Server-Sent Events
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

FIGURE LIST

Figure 2.1	System architecture scheme (<i>Created by AI</i>)
Figure 3.1	AI feed page
Figure 3.2	Social timeline page

TABLE LIST

Table 2.1	Backend event types
-----------	---------------------

ÖZET

Omakase, her gönderinin bir büyük dil modeli (LLM) tarafından gerçek zamanlı olarak üretildiği, kişiselleştirilmiş ve sonsuz kaydırılabilir (infinitely scrollable) bir içerik akışı sunan yerel (native) bir iOS uygulamasıdır. Kullanıcı, ilk katılım aşamasında hobiler, spor, sanat veya bilimsel konular gibi serbest formatta bir dizi ilgi alanı belirler; uygulama ise bu ilgi alanlarına göre özel olarak uyarlanmış kısa, sürükleyici ve sosyal medya tarzında gönderileri sürekli olarak üretir.

Projenin temel teknik zorluğu, akış (streaming) sağlayan bir LLM API'sinin doğası gereği asenkron ve parça parça (token-by-token) gelen çıktısı ile SwiftUI'nin reaktif işleme (rendering) modeli arasında köprü kurmaktır. Bu proje, söz konusu zorluğu iki katmanlı bir mimari aracılığıyla çözmektedir: Gemini modeli çıktısını Sunucu Gönderimli Olaylar (Server-Sent Events - SSE) olarak aktaran bir Python FastAPI arka ucu (backend) ve bu olayları asenkron olarak işleyip akışı gerçek zamanlı güncelleyerek canlı bir "yazıyor (typing)" efekti oluşturan bir SwiftUI iOS istemcisi (client).

Geliştirme sürecinin ikinci aşamasında, projenin tüm temel teknik bileşenlerinin öğrenilmesine ve uygulanmasına odaklanılmıştır. Bu bileşenler; SSE akış veri hattı (streaming pipeline), Gemini API entegrasyonu, Swift eşzamanlılık modeli (async/await ve yapılandırılmış eşzamanlılık / structured concurrency), reaktif durum yönetimi (state management) için SwiftUI Observation çatısı (framework) ve yapay zeka destekli bir ilgi alanı öneri alt sistemini içermektedir. Mevcut durumda tüm bileşenler başarıyla uygulanmış olup uçtan uca çalışır durumdadır.

ABSTRACT

Omakase is a native iOS application that delivers a personalized, infinitely scrollable content feed where every post is generated in real time by a large language model (LLM). The user declares a set of free-form interests during onboarding — such as film directors, board games, or scientific topics — and the application continuously produces short, engaging social-media-style posts tailored to those interests.

The core technical challenge is bridging the inherently asynchronous, token-by-token output of a streaming LLM API with the reactive rendering model of SwiftUI. This project addresses that challenge through a two-layer architecture: a Python FastAPI backend that streams Gemini model output as Server-Sent Events (SSE), and a SwiftUI iOS client that consumes those events asynchronously and updates the feed in real time, producing a live "typing" effect.

The second development phase focused on learning and implementing all key technical components: the SSE streaming pipeline, the Gemini API integration, the Swift concurrency model (async/await + structured concurrency), the SwiftUI Observation framework for reactive state management, and an AI-powered interest suggestion subsystem. All components have been implemented and are functioning end-to-end.

1. INTRODUCTION

This section provides a formal definition of the research topic, establishes its significance within financial markets, defines the project's aim and scope, and reviews the academic literature upon which this study is based. It also defines the core theoretical concepts that serve as the foundation for the proposed methodology.

1.1. Background and Motivation

Mobile content consumption has shifted from curated editorial feeds to algorithmically personalized streams. However, the dominant algorithmic approach depends on massive behavioral datasets collected from millions of users over long periods. This creates a "cold start" problem for new users and limits the degree to which content can be tailored to genuinely niche or specific tastes.

Recent advances in large language models (LLMs) offer an alternative: content can be generated on demand, directly from a natural-language description of the user's interests, eliminating the need for behavioral history. This project explores the feasibility and user experience of an LLM-generated, fully personalized content feed delivered natively to an iOS device over a streaming connection.

The project is also motivated by the engineering challenge: streaming LLM output token-by-token to a mobile client requires careful handling of asynchronous I/O at every layer of the stack — from the model API, through the backend server, across the network, and into the UI rendering pipeline.

1.2. Problem Definition, Aim and Scope

This section outlines the specific problem this project addresses, its formal objective, and its operational boundaries.

1.2.1. Problem Definition

The central engineering problem is to deliver a real-time, token-by-token LLM response stream to an iOS UI with minimal latency and without blocking the main thread. Naively polling a REST endpoint introduces unacceptable latency. A WebSocket-based solution adds bidirectional complexity that is unnecessary for a one-directional data flow. The Server-Sent Events (SSE) protocol is the appropriate tool, but the iOS platform provides no native SSE client; one must be built from URLSession's async byte stream primitives.

1.2.2. Aim of the Study

The formal aim of this study is: To design, implement, and evaluate a full-stack, real-time AI content generation pipeline that streams LLM-generated posts to a native iOS application, producing a seamless "live typing" user experience.

1.2.3. Scope of the Study

The scope of this project is defined as follows:

- **Platform:** Native iOS 26+ application built with SwiftUI and Swift 6 strict concurrency.
- **Backend:** A Python 3.12 FastAPI service deployed locally (and optionally on a cloud host), integrating the Google Gemini API (gemini-2.5-flash model).
- **Content model:** Short-form posts (40–90 words) in eight rotating structural formats (Fun Fact Drop, Hot Take, Rabbit Hole Alert, etc.), ensuring variety across consecutive generations.
- **Personalization:** Interest-based, with an AI-powered suggestion subsystem (POST /interests/suggest) that proposes related and laterally adjacent interests to the user in real time.

1.3. Review of Related Works

This Server-Sent Events (SSE): The SSE specification [1] is a W3C/WHATWG standard that defines a unidirectional, text-based push channel over a persistent HTTP connection. Unlike WebSockets, SSE is strictly server-to-client and operates over standard HTTP/1.1, making it well-suited for streaming LLM tokens where no client-to-server messages are needed after the initial request.

LLM Streaming APIs: OpenAI's chat completions API popularized the convention of streaming token deltas via SSE [2]. Google's Generative AI Python SDK follows the same pattern: calling `generate_content(..., stream=True)` returns a synchronous iterator of chunks, each carrying a partial text payload. Because the Python SDK's streaming API is synchronous, bridging it to an async web framework (FastAPI) requires running the iterator in a worker thread and forwarding chunks to an async queue [3].

FastAPI StreamingResponse: FastAPI's `StreamingResponse` class, backed by Starlette, accepts an async generator and flushes each yielded string to the client immediately. To prevent proxy-level buffering, the `X-Accel-Buffering: no` and `Cache-Control: no-cache` headers must be set [3].

SwiftUI Observation Framework: Apple's `@Observable` macro (introduced in iOS 17) provides fine-grained dependency tracking: only the view properties actually read during a

render pass become dependencies [4]. This is superior to the older `ObservableObject / @Published` approach for a feed use case because appending a token to a single post does not re-render unrelated posts. The `@Observable` `FeedViewModel` and value-type `Post` model used in this project are designed specifically to exploit this property.

Swift Structured Concurrency: Swift's `async/await` system and `Task` API enforce strict actor isolation [5]. The feed streaming work runs in a `Task.detached` context to avoid inheriting the `@MainActor` and blocking the UI. Results are forwarded to the main actor via `await MainActor.run { }` closures, ensuring thread safety without explicit locks.

AI-Powered Onboarding Suggestions: Existing approaches to cold-start personalization rely on collaborative filtering (e.g., "users like you also liked...") [6]. This project takes a generative approach: the Gemini model is prompted with the user's current interests and asked to produce 7 new suggestions, of which ~ 4 are related and ~ 3 are lateral. This requires no user history and produces diverse, specific recommendations (e.g., "Wong Kar-wai" rather than "Arthouse Cinema").

2. PROPOSED METHODOLOGY

This section outlines the proposed technique for the design project.

2.1. Key Concepts And Theoretical Framework

Server-Sent Events Protocol. An SSE stream is a text/event-stream HTTP response body consisting of zero or more event frames, each terminated by a blank line (`\n\n`). Each frame may contain `event:`, `data:`, `id:`, and `retry:` fields. In this project, four named event types are used:

Table 2.1 Backend event types

Event	Payload	Description
start	<code>{ "id": "<uuid>" }</code>	Signals the beginning of a new post
title	<code>{ "title": "<headline>" }</code>	Delivers the post's headline
token	<code>{ "text": "<delta>" }</code>	Delivers one body text chunk
done	<code>{ "id": "<uuid>" }</code>	Signals the post is complete
error	<code>{ "message": "<text>" }</code>	Reports a server-side failure

Post Format Rotation: To prevent consecutive posts from feeling structurally identical, the backend selects one of eight named post formats based on `seen_count mod 8`. The format name and its prose instruction are injected into the user prompt. The format label is never included in the generated output. The eight formats are: Fun Fact Drop, Hot Take, Rabbit Hole Alert, Unlikely Connection, Tiny Recommendation, Reframe, The Thing Nobody Talks About, and Cursed Trivia.

Interest Suggestion Algorithm: The `/interests/suggest` endpoint receives: (a) interests the user has already added, (b) an optional partial string the user is currently typing (draft), and (c) a list of suggestions already shown (`exclude_suggestions`). The Gemini model is prompted with a strict system instruction to return a JSON array of exactly 7 strings, ~4

related and ~ 3 lateral, never repeating an existing interest or excluded suggestion. Debouncing (600 ms) on the client side prevents excessive API calls while the user is typing.

Title Parsing from Stream: The Gemini model is instructed to output its response in a fixed two-part format: the first line must be TITLE: <headline>, followed by the post body. The backend's `_feed_tokens_from_stream_chunks` function buffers stream chunks until the first newline is encountered, extracts and validates the title line with a regular expression, emits it as a title SSE event, and then forwards all subsequent chunks as token events. This ensures the iOS client can display the headline separately from the scrolling body text.

2.2. System Architecture

The overall architecture is a client-server model with a single bidirectional boundary:

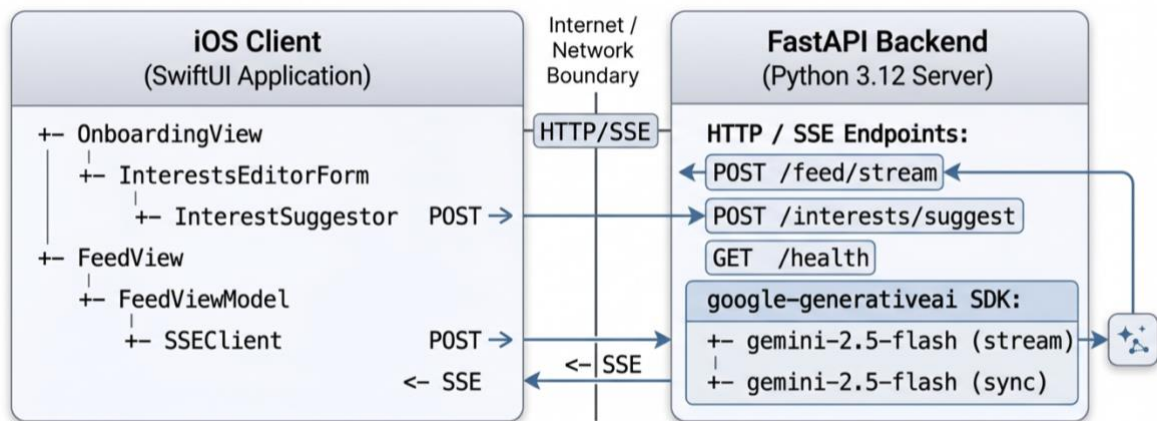


Figure 2.1 System architecture scheme

2.3 Component Design

Backend components:

FeedRequest / SuggestRequest (Pydantic models): Validated request schemas that enforce types and provide defaults. `FeedRequest` carries `interests: list[str]` and `seen_count: int`.

_build_user_prompt(): Assembles the user turn of the Gemini prompt. Selects the appropriate format template from `_POST_FORMATS` based on `seen_count % 8` and constructs a human-readable interest list.

_feed_tokens_from_stream_chunks(): Runs synchronously in a worker thread. Buffers chunks, parses the TITLE: line, and forwards ("title", str) and ("token", str) tuples to an asyncio.Queue using loop.call_soon_threadsafe.

_stream_post(): Async generator consumed by StreamingResponse. Drains the queue and yields SSE-framed JSON strings.

interests_suggest(): Standard async endpoint. Calls the Gemini model synchronously via asyncio.to_thread, parses the JSON array response through _parse_suggestion_strings() (with markdown-fence and bracket-extraction fallbacks), and returns { "suggestions": [...] }.

iOS client components:

SSEClient (enum): Stateless SSE parser. Opens a URLSession.AsyncBytes stream, accumulates bytes in a Data buffer, and pops complete SSE frames whenever a \n\n or \r\n\r\n delimiter is found. Each frame is parsed into an SSEEvent(event:data:) value and yielded from an AsyncThrowingStream.

FeedViewModel (@Observable, @MainActor): Owns the posts: [Post] array. Launches a Task.detached producer that iterates the SSEClient stream and dispatches mutations back to the main actor. Handles stream cancellation gracefully: if a new post is requested while a previous stream is in flight, the old task is cancelled and any incomplete empty post is removed.

Post (struct): Value type with id: UUID, title: String, text: String, and isComplete: Bool. Stored in a [Post] array; mutations are performed by replacing the entire array to ensure @Observable detects the change.

InterestSuggestor (enum): Calls POST /interests/suggest, decodes the JSON response, and returns an InterestSuggestResponse value. Handles HTTP error codes, unexpected formats, and network errors without throwing, returning a loadingIssue description instead.

InterestsEditorForm (SwiftUI view): Shared between onboarding and the "Adjust Tastes" sheet. Maintains a 600 ms debounce on both draft text and interests array changes. The refresh button passes the currently displayed suggestions as exclude_suggestions to guarantee a fresh set.

3. IMPLEMENTATION AND EXPERIMENTAL SETUP

This chapter describes the practical steps taken to development of the application.

3.1 Development Environment and Tools

The project uses the following environment:

Backend: Python 3.12, FastAPI 0.115, Uvicorn 0.34, google-generativeai 0.8, python-dotenv, Pydantic v2. Dependency management via pip with a requirements.txt and a .venv virtual environment.

iOS Client: Xcode 26 Beta, Swift 6, SwiftUI, SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor. No third-party Swift packages; all networking uses URLSession.

AI Model: Google Gemini gemini-2.5-flash via the google-generativeai Python SDK. The model name is configurable via the GEMINI_MODEL environment variable.

Transport: HTTP/1.1 with text/event-stream content type for the feed endpoint; standard JSON REST for the suggestion endpoint.

3.2 Backend Implementation

Gemini streaming bridge: The synchronous `generate_content(..., stream=True)` call is executed in a worker thread via `loop.call_soon_threadsafe` and `asyncio.to_thread`. Each chunk's text is extracted through a two-tier helper (`_stream_chunk_text`) that first attempts `chunk.text` and falls back to iterating `chunk.candidates[0].content.parts` to handle SDK version differences.

Title parsing: The function `_feed_tokens_from_stream_chunks` maintains a `carry` buffer and a `title_done` boolean flag. It delays emitting any token events until the first `\n` is seen. The first line is matched against the regex `(?i)^\s*TITLE:\s*(.+)$`; if it matches, the captured group is sent as a `("title", str)` queue item; if it does not match, the line is prepended back to `carry` and body emission begins immediately (defensive fallback). The following code excerpt illustrates the core logic:

```
if not title_done:
    if "\n" not in carry:
        continue
    first_line, carry = carry.split("\n", 1)
```



```

m = _TITLE_LINE.match(first_line.strip())
if m:
    loop.call_soon_threadsafe(queue.put_nowait, ("title", m.group(1).strip()))
else:
    carry = first_line + "\n" + carry
title_done = True

```

SSE framing: The `_sse_event(data, event=None)` helper serializes a Python dict to JSON and formats it as an SSE frame: an optional event: line, a data: line, and two trailing newlines. `ensure_ascii=False` is set so non-ASCII interest strings (e.g., Japanese, Arabic) are preserved.

Interest suggestion parser: `_parse_suggestion_strings` applies three progressive parsing strategies: (1) direct `json.loads` of the response after stripping markdown fences, (2) extracting the first [...] substring and parsing it, (3) regex extraction of double-quoted strings as a last resort. This robustness layer is necessary because LLMs occasionally wrap JSON output in prose or markdown blocks despite explicit instructions not to.

CORS and buffering headers: The backend adds `CORSMiddleware` with permissive settings for local development. The `StreamingResponse` is sent with `Cache-Control: no-cache`, `Connection: keep-alive`, and `X-Accel-Buffering: no` to prevent any reverse proxy from buffering the stream.

3.3 iOS Client Implementation

SSEClient frame assembly: Rather than splitting on newlines and tracking state across lines, `SSEClient` accumulates raw bytes in a Data buffer and pops complete frames by searching for the `\n\n` or `\r\n\r\n` delimiter sequences. This byte-level approach is immune to UTF-8 multi-byte character boundaries and handles both Unix and Windows line endings. The `extractNextSseFrame` function checks both delimiter variants and picks the one that appears first in the buffer.

FeedViewModel concurrency model: The `@Observable @MainActor` view model exposes `posts`, `isGenerating`, and `errorMessage`. Post streaming is performed in a `Task.detached(priority: .userInitiated)` closure that is explicitly not isolated to the main actor, allowing `URLSession.AsyncBytes` iteration to proceed without blocking the UI. Each mutation (append title, append token text, mark complete) is dispatched via `await MainActor.run { [weak self] in ... }`. A `activeStreamPostID` property ensures that only the task that created a given post can clear the `isGenerating` flag, preventing race conditions when the user requests a new post before the previous stream finishes.

Post state mutation: Because `Post` is a value type (struct) stored in `[Post]`, SwiftUI's `@Observable` only observes changes to the array itself, not to individual elements. Each mutation therefore follows the copy-replace pattern:

```
private func appendText(_ text: String, to postID: UUID) {
    guard let idx = posts.firstIndex(where: { $0.id == postID }) else { return }
    var copy = posts
    copy[idx].text.append(contentsOf: text)
    posts = copy // whole-array assignment triggers @Observable
}
```

InterestsEditorForm debounce: The suggestion fetch is scheduled via a debounceTask that is cancelled and re-created on every change to draft or interests. After a 600 ms Task.sleep, the current snapshot of interests and draft text is captured and passed to InterestSuggestor.suggest. The "Refresh Ideas" button cancels any pending debounce, passes the currently displayed suggestions as exclude_suggestions, and immediately triggers a new fetch.

ATS configuration: iOS App Transport Security blocks plain HTTP by default. For simulator development, NSAllowsLocalNetworking = true is added to the app's Info.plist. The backend URL defaults to http://127.0.0.1:8000 and is overridable at runtime via the OMAKASE_API_URL Info.plist key, allowing the same binary to be pointed at a LAN IP address for physical device testing.

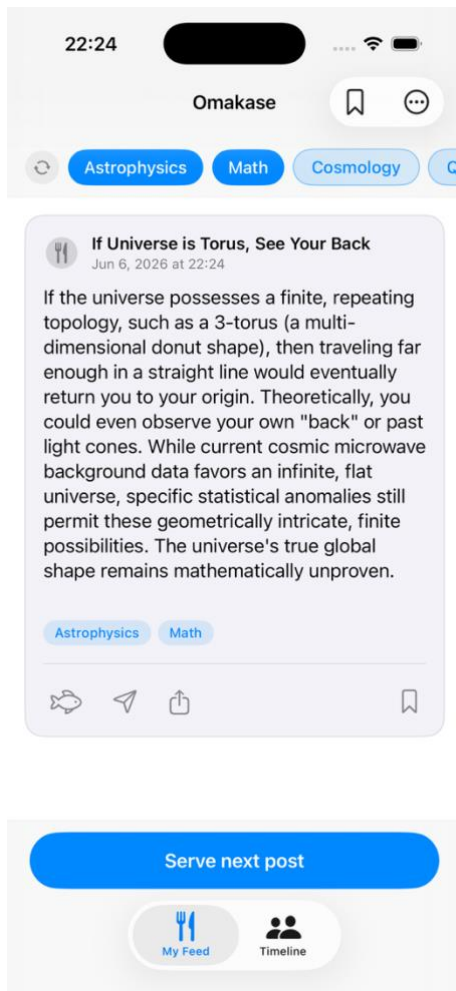


Figure 3.1 AI feed page

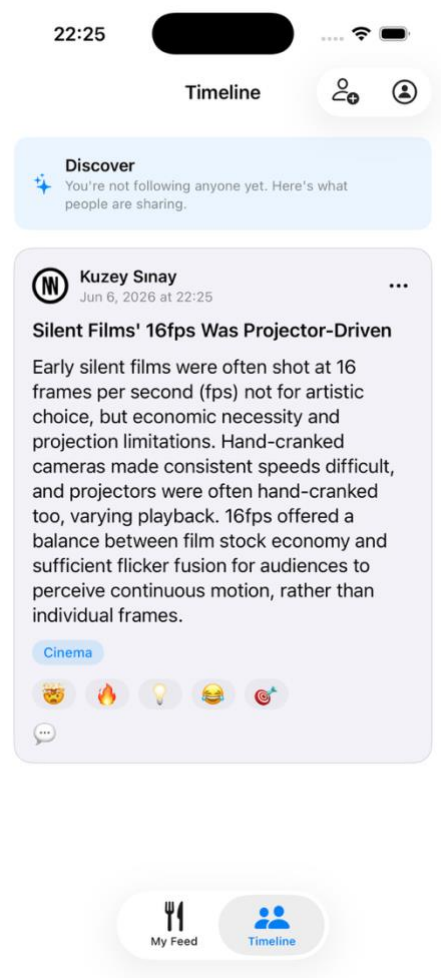


Figure 3.2 Social timeline page

4. CURRENT PROGRESS AND NEXT STEPS

4.1. Achievements

End-to-end streaming pipeline: The complete path from user interest input to Gemini streaming to SSE framing to SSEClient parsing to FeedViewModel state update to SwiftUI rendering is fully implemented and functional.

Real-time typing effect: The iOS feed displays a blinking cursor while a post is generating and appends each token chunk as it arrives, producing a live typing animation without any UI jank.

Title / body separation: The backend reliably extracts the TITLE: line from the stream and emits it as a separate SSE event, allowing the iOS client to display the headline in a

distinct typographic style before the body begins to appear.

Post format rotation: Eight structurally distinct post formats are cycled based on the session's `seen_count`, ensuring that consecutive posts feel varied in tone and structure.

AI interest suggestion: The POST `/interests/suggest` endpoint and its iOS `InterestSuggestor` client are fully implemented. Suggestions appear within approximately 1 second of the user pausing their input, can be refreshed for an entirely new set, and automatically exclude already-added interests from the displayed chips.

Onboarding flow: OnboardingView to InterestsEditorForm to Feed navigation is complete. Interests are persisted via `@AppStorage` and survive app restarts.

Bookmark subsystem: Posts can be bookmarked from the feed; `BookmarkStore` persists bookmarks to `UserDefaults` and a dedicated `BookmarksSheet` lists them.

Robust error handling: Network failures, missing API keys, model errors, and stream-with-no-output scenarios all produce human-readable error messages surfaced in the UI. Cancelled streams clean up incomplete posts without leaving stuck loading states.

4.2. Future Work

The remaining development effort will focus on the following areas:

Deployment: Packaging the FastAPI backend for cloud deployment (e.g., Cloud Run or Railway) so the app can be tested on a physical iPhone without a local Mac running the server.

UI polish: Refining card animations (entrance transitions, skeleton loading state), adding haptic feedback on stream completion, and implementing pull-to-refresh with a "serve next post" affordance.

Interests management: Allowing the user to edit interests from within the feed (via the existing `AdjustTastesSheet`) and immediately refreshing the prompt for the next post.

Evaluation: Conducting a small user study to assess perceived content quality, variety, and the subjective experience of the live-typing effect compared to a static response.

Final report: Writing the comprehensive final (third) progress report and preparing the project demonstration.

REFERENCES

- [1] WHATWG, (2024), “Server-Sent Events Living Standard”, <https://html.spec.whatwg.org/multipage/server-sent-events.html>.
- [2] OpenAI, (2024), “Streaming API reference”, <https://platform.openai.com/docs/api-reference/streaming>.
- [3] Ramírez, S., (2024), “FastAPI: StreamingResponse documentation”, <https://fastapi.tiangolo.com/advanced/custom-response/#streamingresponse>.
- [4] Apple Inc., (2024), “Observation framework — Swift documentation”, <https://developer.apple.com/documentation/observation>.

- [5] Apple Inc., (2024), “Swift concurrency: Actors and async/await”, <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/concurrency/>.
- [6] Linden, G., Smith, B. and York, J., (2003), “Amazon.com recommendations: Item-to-item collaborative filtering”, IEEE Internet Computing, Vol. 7, No. 1, pp. 76-80.
- [7] Google, (2024), “Gemini API — Streaming responses”, <https://ai.google.dev/api/generate-content#stream>.

Resume

Name Surname	Kuzey SINAY
Date of Birth	12.03.2004
Place of Birth	Manisa
High School	2018 – 2022 Akhisar Fen Lisesi
Internships	Turkcell Communication Services – (4 Weeks) Borusan Otomotiv – (Current)